A1 Analytic IK + Per-Leg Crawl Gait — Technical Documentation

Modules covered: `IK_A1.py` (the library), `IK_module_test.py` (demo/test harness), `circle_wlk_test.py` (circle-walking script)
Goal: write your own scripts in which **every leg has its own step height, step length in x, step length in y** (and related parameters), changing per gait cycle.

---

1. Architecture at a glance

```
                ┌──────────────────────────────┐
your script ──► │ CycleCommand                 │  what the gait should do this cycle
                │   legs{FR,FL,RR,RL: LegStep}│  per-leg length_x / length_y / height
                │   cycle_time, stance_z,      │  global: timing + posture
                │   swing_frac, cg_offset_x,   │
                │   phase_offset{...}          │  per-leg timing
                └──────────────┬───────────────┘
                               ▼
                ┌──────────────────────────────┐
                │ CrawlGait.update(dt)         │  advances phase, builds foot paths
                │  → foot_targets  (x,y,z)     │
                └──────────────┬───────────────┘
                               ▼
                ┌──────────────────────────────┐
                │ foot_to_joints → leg_ik      │  analytic IK
                └──────────────┬───────────────┘
                               ▼
            {leg: (q1 hip, q2 thigh, q3 calf)}  → your simulator / motors
```

The library is **pure math** (no PyBullet import). PyBullet only appears in the test scripts, so you can unit-test gaits headlessly (Section 10.4).

---

2. Project layout and imports

Both test scripts add the *parent of their own folder* to `sys.path` and import the library as a package module:

```
root_dir = str(Path(__file__).resolve().parent.parent)   # folder ABOVE the script's folder
sys.path.append(root_dir)
from Inverse_Kinematics.IK_A1 import CrawlGait, CycleCommand, LegStep, ...
```

So the expected layout is:

```
project/
└── Inverse_Kinematics/
    ├── IK_A1.py
    ├── IK_module_test.py
    ├── circle_wlk_test.py
    └── your_script.py
```

`circle_wlk_test.py` also does `from IK_module_test import ...`, which works because the script's own folder is `sys.path[0]` when run directly. If you put your script elsewhere, either mirror this header or add an `__init__.py` and use a package import.

> ***Note on stale names.*** *Docstrings and printed hints refer to `a1_ik.py` / `walk_test.py` / `walk_circle.py`. These are the earlier file names; the actual files are `IK_A1.py`, `IK_module_test.py`, `circle_wlk_test.py`. When a message says "flip `IK_X_SIGN` in a1_ik.py", it means `IK_A1.py`.*

Minimal header for your own script:

```
import sys, math
from pathlib import Path
sys.path.append(str(Path(__file__).resolve().parent.parent))
from Inverse_Kinematics.IK_A1 import (
    LEG_NAMES, SIDE, L1, MAX_REACH,
    CrawlGait, CycleCommand, LegStep,
    foot_to_joints, leg_ik, leg_fk, IK_X_SIGN,
)
```

---

3. Coordinate frames and sign conventions

3.1 Public (physical) frame — what you pass in

Hip-relative body frame: **+x forward, +y left, +z up**. The origin is each leg's own hip.

Parameter	Positive value means
`length_x`	leg walks the body **forward** (negative → backward)
`length_y`	leg walks the body **to the left** (negative → right)
`height`	swing-foot clearance **above** the stance plane

Why "walks the body forward"? During stance the foot sweeps from the front to the back of its stride (`+L/2 → −L/2`) relative to the body, which pushes the body forward. During swing it returns from back to front through the air. The same logic gives lateral motion: a foot dragged toward −y pushes the body toward +y.

3.2 Internal IK frame and `IK_X_SIGN`

`leg_ik()` keeps the original script's math, whose x-axis is mirrored relative to the physical frame. The mirror is absorbed in one place:

```
IK_X_SIGN = -1
def foot_to_joints(leg, dx, dy, z):
    side = SIDE[leg]
    return leg_ik(IK_X_SIGN * dx, side * L1 + dy, z, side)
```

Calibration test: run `IK_module_test.py --mode forward --direct`. If it prints `=> moved the WRONG way`, set `IK_X_SIGN = +1`. Do this once per URDF, not per script.

3.3 The `dy` offset and `SIDE`

`SIDE = {"FR": -1, "FL": 1, "RR": -1, "RL": 1}`. Inside `foot_to_joints` the IK y-coordinate is `side * L1 + dy`, i.e. `dy = 0` **means the foot sits directly below the hip abduction offset** (`L1 = 0.083165 m` outboard of the hip axis). Your `length_y` therefore moves the foot symmetrically around the natural outboard position, on both left and right legs, with **+y = left for all four legs** (the sign is not mirrored per side — the mirroring is handled by `SIDE`).

---

4. Kinematic constants and the reachable workspace

```
L1 = 0.083165                              # hip abduction offset
L2 = sqrt(0.003237² + 0.022327² + 0.172674²)   # thigh ≈ 0.174 m
L3 = sqrt(0.006435² + 0.092612²)               # calf  ≈ 0.093 m
MAX_REACH = L2 + L3                            # ≈ 0.267 m
```

(Print them if you need exact values; they are derived from the URDF parent-frame offsets.)

IK procedure (`leg_ik`)

1. `d = max(0, y² + z² − l1²)`, `b = −√d` — the vertical extent of the leg in its pitch plane.
2. `q1 = atan2(z, y) − atan2(b, l1s)` — hip ab/ad.

3. In the pitch plane, `a = x`, `r² = a² + b²`, **clamped to** `(l2+l3)²`.
4. `cos q3` from the law of cosines, clamped to `[−1, 1]`; `q3 = −acos(...)` (knee bends the A1 way — always the same solution branch).
5. `q2 = atan2(a, −b) − atan2(l3 sin q3, l2 + l3 cos q3)`.

4.1 The silent clamp — the most important intricacy

`leg_ik` **never raises**. If a foot target is out of reach it silently clamps to the workspace boundary and returns joints for the *closest reachable* point. The symptom is that the foot stops following the commanded path (strides look shorter than requested, or the leg "hits a wall" during swing).

Approximate x-reach at the default stance depth (`z = −0.22`, `dy = 0`): `|x| ≲ √(0.267² − 0.22²) ≈ 0.15 m`. Your foot's x-extent during a cycle is:

```
x ∈ [ cg_offset_x − |length_x|/2 ,  cg_offset_x + |length_x|/2 ]
```

With defaults (`cg = −0.085`, `length_x = 0.08`): `[−0.125, −0.045]` — comfortably inside. With `length_x = 0.20` you'd reach `−0.185` and clamp.

Swing raises the foot to `z = stance_z + height`. Keep the foot from getting too close to the hip (the leg folds; practical limit is around `height ≈ 0.12–0.14` with `stance_z = −0.22`). Use the verification helper in Section 10.4 instead of guessing.

---

5. Data structures

5.1 `LegStep` — the per-leg parameters

```python
@dataclass(frozen=True)
class LegStep:
    length_x: float = 0.08   # fore-aft stride  (+ forward)
    length_y: float = 0.0    # lateral stride   (+ left)
    height:   float = 0.10   # swing clearance above stance plane
```

Frozen → immutable. To modify, create a new one (or `dataclasses.replace`).

Semantics:

- `length_x`, `length_y` are **peak-to-peak stride lengths** of the foot relative to the body during stance (foot travels `+L/2 → −L/2`). They can be combined — the foot then travels along a diagonal line.
- `length_x = length_y = 0` → the leg **steps in place** (lifts and replaces the foot, still lifting by `height`).
- `height` is added as `height · sin(πs)` to the swing trajectory, so it is a *peak* clearance, reached mid-swing. `height ≥ 0` — a negative value would drive the foot into the ground.
- Strides can differ per leg *and be different signs* per leg (this is how rotation-in-place and circular paths work).

5.2 `CycleCommand` — one gait cycle's specification

Field	Default	Scope	Meaning
`legs`	required	**per leg**	`{ "FR": LegStep, "FL": …, "RR": …, "RL": … }` — all four required
`cycle_time`	1.0 s	global	duration of one full cycle
`stance_z`	−0.22 m	global	nominal foot depth below hip (must be in `(−MAX_REACH, 0)`)
`swing_frac`	0.15	global	fraction of a cycle each leg spends in swing
`cg_offset_x`	−0.085 m	global	static x-shift of all feet (negative = feet behind hips → trunk shifts forward over the support polygon)
`phase_offset`	RL 0, FR .25, RR .5, FL .75	**per leg**	phase shift of each leg's cycle

`__post_init__` validates: all four legs present in `legs` and `phase_offset`; `cycle_time > 0`; `0 < swing_frac < 1`; `-MAX_REACH < stance_z < 0`. It does **not** validate reach of the strides (see 4.1) nor the crawl property (`swing_frac ≤ 0.25`, see 6.2).

> ***What is not per-leg:*** *`stance_z`, `cg_offset_x`, `swing_frac`, `cycle_time`. These are global to the cycle. Per-leg = `length_x`, `length_y`, `height`, and `phase_offset`. If you need per-leg stance height (e.g. leveling on a slope), you must extend `_LegState`/`_latch` (Section 11).*

5.3 Constructors and helpers

```
# Same LegStep on all legs
CycleCommand.uniform(length_x=0.08, length_y=0.0, height=0.10, **global_kw)
# Skid-steer turn: turn ∈ [-1, 1]; +1 = left. Right legs get length_x*(1+turn),
# left legs get length_x*(1-turn). length_y and height are shared.
CycleCommand.differential(length_x=0.08, turn=0.0, height=0.10, length_y=0.0, **global_kw)
# Copy with one leg changed (immutable style)
cmd2 = cmd.with_leg("FL", height=0.14)          # any LegStep field(s)
```

`**global_kw` accepts `cycle_time`, `stance_z`, `swing_frac`, `cg_offset_x`, `phase_offset`.

`with_leg` chains, so you can build a fully custom command from a base:

```
cmd = (CycleCommand.uniform(0.08, 0.0, 0.10)
       .with_leg("FL", height=0.14)
       .with_leg("RR", length_x=0.06, length_y=0.01))
```

Caveat: `with_leg` uses `dataclasses.replace`, so the new command **shares the same `phase_offset` dict object** with the original. Don't mutate `cmd.phase_offset[...]` in place after copying — build a new dict instead.

5.4 Building a command with total per-leg control

The most explicit form (what the circle script does):

```
legs = {
    "FR": LegStep(length_x=0.09, length_y=+0.02, height=0.12),
    "FL": LegStep(length_x=0.05, length_y=-0.02, height=0.10),
    "RR": LegStep(length_x=0.09, length_y=-0.01, height=0.10),
    "RL": LegStep(length_x=0.05, length_y=+0.01, height=0.11),
}
cmd = CycleCommand(
    legs=legs,
    cycle_time=1.0,
    stance_z=-0.22,
    swing_frac=0.15,
    cg_offset_x=-0.085,
)
```

---

6. The gait engine (`CrawlGait`)

6.1 Construction and the update loop

```
gait = CrawlGait(command, cycle_provider=None, blend_time=0.5)
joints = gait.update(dt)     # {leg: (q1, q2, q3)}, call once per sim tick
```

Useful attributes: `gait.t` (elapsed time), `gait.cycle_index`, `gait.cycle_phase ∈ [0,1)`, `gait.cmd` (currently active command), `gait.foot_targets` (`{leg: (x,y,z)}` physical frame), `gait.joint_targets`, and `gait.swinging` (list of legs currently airborne).

> `update()` *returns the **same dict object** every call (mutated in place). If you log or store it, copy it:* `dict(gait.foot_targets)`*; the module's self-test does exactly this.*

6.2 Phase, swing windows and the crawl property

Every leg has a local phase `lp = (cycle_phase + phase_offset[leg]) % 1`.

- `lp < swing_frac` → **swing** (foot in the air)
- otherwise → **stance**, with `s = (lp − swing_frac)/(1 − swing_frac)` running 0→1

With default offsets the swing windows in *cycle phase* are:

```
cycle phase   0.00        0.25        0.50        0.75        1.00
              ├──RL──┤    ├──FL──┤    ├──RR──┤    ├──FR──┤
                 sf          sf          sf          sf      (sf = swing_frac)
```

Order in time: **RL → FL → RR → FR**. Because each window has width `swing_frac` and windows start every 0.25, `swing_frac ≤ 0.25` **guarantees at most one leg airborne** (the self-test asserts this: `len(g.swinging) <= 1`). Above 0.25, windows overlap, two legs lift together and the crawl's static stability is lost. The constructor does not enforce this.

The gait starts at `cycle_phase = swing_frac` ("just after RL's swing window so no leg begins mid-swing"), and every leg's `pos` and `swing_from` are initialised on its stance line for that phase.

6.3 Foot trajectories

Stance (`_stance_pos`), with `k = 0.5 − s`:

```
x = cg + length_x · k
y =      length_y · k
z = stance_z
```

A straight line from `(cg + Lx/2, Ly/2)` to `(cg − Lx/2, −Ly/2)` at constant speed.

Swing (`_swing_pos`), with `e = ½(1 − cos πs)`:

```
x = fx + (tx − fx)·e            tx = cg + Lx/2
y = fy + (ty − fy)·e            ty = Ly/2
z = fz + (tz − fz)·e + height·sin(πs)
```

`(fx, fy, fz) = swing_from` is the foot position **at the moment swing began**. The cosine easing gives zero velocity at lift-off and touch-down (no impact spike); the sine term is the clearance arc.

6.4 Timing, latching and continuity (the core intricacy)

- A new command becomes active only at a **cycle boundary** (`cycle_phase` wraps past 1.0). Nothing changes mid-cycle.
- **Each leg latches its parameters (`LegStep`, `stance_z`, `cg_offset_x`) at the start of its own swing** — not at the cycle boundary. Consequently:
  - a leg keeps executing the *old* step through the remainder of its stance even though a new command is active;
  - RL swings at phase 0 so it picks up the new command immediately; FL, RR, FR follow ¼, ½, ¾ cycle later. **A change therefore fully takes effect only after one whole cycle**, and during that transition the legs run a mixture of old and new strides. Plan for it (e.g. a gentle ramp, Section 10.3).
- Swing starts from wherever the foot **actually is** (`swing_from = st.pos`), and the latched stance line always starts at the new `cg + Lx/2`. So changing lengths, heights or even `cg_offset_x` between cycles never causes a foot jump.
- **Not latched:** `cycle_time`, `swing_frac`, `phase_offset` are read live from `gait.cmd`. Changing `swing_frac` or `phase_offset` between cycles will change `s`/`lp` immediately for legs in stance, causing a **transient jump** (the docstring warns about `phase_offset`; `swing_frac` behaves the same). Change these rarely, and ideally only with the robot standing still.

6.5 Providing commands each cycle

Two mechanisms, both applied at the next cycle boundary:

```
gait = CrawlGait(provider(0), cycle_provider=provider)   # pull: provider(cycle_index)
gait.set_next_cycle(cmd)                                  # push: one-shot, higher priority
```

- `cycle_provider(n)` is called at each boundary with the new `cycle_index`. Return a `CycleCommand` or `None` (= keep the current one).
- `set_next_cycle(cmd)` queues a command for the next boundary. If a command is pending, the provider is **not** called at that boundary. The pending slot is consumed (cleared) once used.
- Initial command: pass it as the first argument; the provider is only consulted from the first boundary on. That's why the demos use `CrawlGait(provider(0), cycle_provider=provider)`.
- If `dt` is large enough to cross several boundaries in one `update`, `_begin_cycle` runs once per boundary crossed.

6.6 Start-up blend

`blend_time = 0.5` s: the output foot target is `neutral + w·(pos − neutral)` with `w = smoothstep(t / blend_time)`. `neutral = (cg_offset_x, 0, stance_z)` of the **command passed to the constructor** (not updated later — irrelevant after the blend ends). Set `blend_time=0` to disable (`w = 1`). Use blending when starting the gait from a stand pose; disable it in headless tests that inspect the raw trajectory.

---

7. Relating step parameters to body motion

During stance a leg's foot moves `Lx` in `(1 − swing_frac)·T` seconds; in a crawl every leg does this, so the commanded body velocity is:

```
vx = length_x / ((1 − swing_frac) · cycle_time)
vy = length_y / ((1 − swing_frac) · cycle_time)
```

Defaults: `0.08 / 0.85 ≈ 0.094 m/s`. Mean horizontal foot speed in swing is much larger (`Lx` in `swing_frac·T = 0.15 s` ≈ 0.5 m/s), so shortening `cycle_time` quickly saturates the motors.

If the four legs get **different** `length_x`, they are asking the body for different velocities. Real ground contact reconciles it: the body yaws (e.g. longer strides on the right → turns left, as in `differential`). This is the mechanism behind steering, and also why one mis-tuned leg causes drift.

---

8. Reference: how the provided scripts use the module

8.1 `IK_module_test.py`

- **Stand-up:** `crouch_targets(z)` calls `foot_to_joints(leg, CG_X, 0, z)` for each leg; `stand_up` cosine-interpolates `z` from `Z_DOWN (−0.10)` to `Z_STAND (−0.22)`.
- **Constants that must agree with the gait:** `Z_STAND == CycleCommand.stance_z` and `CG_X == CycleCommand.cg_offset_x`. If they differ, the robot stands in one posture and the gait begins in another (blend hides part of it, but the mismatch is real).
- **Demos** are functions `n -> CycleCommand`:

Mode	Command
`forward` / `backward`	`uniform(length_x=±0.08)`
`sidestep`	`uniform(length_x=0, length_y=0.05)`
`turn`	`differential(0.08, turn=0.6)`
`custom`	`uniform` + `.with_leg("FL", height=…)` + `.with_leg("RR", length_x=0.06, length_y=0.01)`
`sequence`	cycles through forward → drift left → turn → sidestep → backward, 4 cycles each

- **Main loop:** `command_legs(robot, joints, gait.update(DT))` then `step()`; `command_legs` applies `POSITION_CONTROL` with `force = MAX_FORCE (33.5 N·m)`.

- **Failure check:** base height < 0.12 m is treated as a fall.

```
python IK_module_test.py --urdf urdf/a1_unitree.urdf --mode custom
python IK_module_test.py --mode sequence --duration 40
python IK_module_test.py --mode sidestep --direct
```

8.2 `circle_wlk_test.py` — a full per-leg example

`circle_command(radius, speed, ...)` computes **different** `length_x` **and** `length_y` **for every leg** using rigid-body kinematics.

For a body rotating about a centre at `(0, R)` (left of the robot for `R > 0`) with yaw rate `ω = v/R`, a foot at `(fx, fy)` measured from the *hip-centroid* must have body-frame velocity

```
r = (fx, fy − R)
v_foot = ω × r = (−ω(fy − R), ω fx) = (v − ω fy,  ω fx)        (since ωR = v)
```

```
stride_x = (v − ω·fy) · stance_t       outer legs longer, inner legs shorter
stride_y = (ω·fx)     · stance_t       front legs sidestep one way, rear the other
stance_t = (1 − swing_frac) · cycle_time
```

The neutral foot position uses `fx = hip_x + CG_X − centre_x`, `fy = hip_y + SIDE·L1 − centre_y`, with hip positions `(±0.1805, ±…)` and the centre taken as the centroid of the four hips.

Uniform limiting: if any stride exceeds `max_lx = 0.10` or `max_ly = 0.05`, *all* strides are multiplied by the same factor. This preserves the ratio between legs (so the radius stays correct) while reducing speed — the function returns the *effective* speed. Never clip strides individually; that would change the radius.

The returned command is reused every cycle: `CrawlGait(cmd, cycle_provider=lambda n: cmd)`. Negative `radius` gives a clockwise circle (ω < 0, signs flip automatically).

```
python circle_wlk_test.py --radius 0.6
python circle_wlk_test.py --radius -0.6 --direct --laps 1
```

Headless runs finish with a Kasa least-squares circle fit and compare radius/centre against the command. If the fitted centre is on the wrong side, revisit `IK_X_SIGN` and the sign of `length_y`.

---

9. Quick-start: your first per-leg script

```
import sys, math, time
from pathlib import Path
import pybullet as p, pybullet_data
sys.path.append(str(Path(__file__).resolve().parent.parent))
from Inverse_Kinematics.IK_A1 import LEG_NAMES, CrawlGait, CycleCommand, LegStep
from IK_module_test import (DT, leg_joint_map, command_legs, crouch_targets,
                            step, stand_up, Z_DOWN)
# 1) sim + robot ------------------------------------------------------
p.connect(p.GUI)
p.setAdditionalSearchPath(pybullet_data.getDataPath())
p.setGravity(0, 0, -9.81); p.setTimeStep(DT)
p.loadURDF("plane.urdf")
robot  = p.loadURDF("urdf/a1_unitree.urdf", [0, 0, abs(Z_DOWN) + 0.02])
joints = leg_joint_map(robot)
for j in range(p.getNumJoints(robot)):
    p.setJointMotorControl2(robot, j, p.VELOCITY_CONTROL, force=0)   # free default motors
# 2) crouch, then stand -----------------------------------------------
command_legs(robot, joints, crouch_targets(Z_DOWN), reset=True)
for _ in range(int(0.3 / DT)): step(True)
stand_up(robot, joints, realtime=True)
# 3) per-leg command, per-cycle provider ------------------------------
def my_cycle(n):
    return CycleCommand(
        legs={
```

```
            "FR": LegStep(length_x=0.08, length_y=0.00, height=0.12),  # lifts higher
            "FL": LegStep(length_x=0.08, length_y=0.00, height=0.10),
            "RR": LegStep(length_x=0.07, length_y=0.01, height=0.10),
            "RL": LegStep(length_x=0.07, length_y=0.01, height=0.10),
        },
        cycle_time=1.0, stance_z=-0.22, swing_frac=0.15, cg_offset_x=-0.085,
    )
gait = CrawlGait(my_cycle(0), cycle_provider=my_cycle)
# 4) main loop ------------------------------------------------------------
for _ in range(int(20 / DT)):
    command_legs(robot, joints, gait.update(DT))
    step(True)
```

---

10. Recipes

10.1 Different height per leg (obstacle / stair clearance)

```
def stair_cycle(n):
    base = CycleCommand.uniform(length_x=0.06, height=0.08)
    # front legs must clear the step, rear legs follow the same edge later
    return (base.with_leg("FR", height=0.13)
                .with_leg("FL", height=0.13))
```

Watch the reach limit: `stance_z + height` must stay reachable. The IK clamps silently; check with 10.4.

10.2 Compensating a weak or drifting leg

If the robot drifts left, the right-side legs are pushing too little or the left too much. Adjust per-leg stride:

```
def trimmed(n, trim=0.01):
    base = CycleCommand.uniform(length_x=0.08)
    return (base.with_leg("FL", length_x=0.08 - trim)
                .with_leg("RL", length_x=0.08 - trim))
```

Confirm drift/yaw with the base yaw printout from the harness rather than by eye.

10.3 Ramping strides smoothly (avoid the mixed-stride transient)

Because a command takes a full cycle to propagate to all four legs (Section 6.4), large changes produce uneven pushes. Ramp instead:

```
def ramped(n):
    a = min(1.0, n / 6)                         # 0 → 1 over 6 cycles
    return CycleCommand.uniform(length_x=0.08 * a, height=0.10)
```

Or, for arbitrary targets, slew per cycle:

```
class Slew:
    def __init__(self, target, rate=0.01):
        self.cur = dict.fromkeys(LEG_NAMES, 0.0)
        self.target, self.rate = target, rate      # target: {leg: length_x}
    def __call__(self, n):
        for l in LEG_NAMES:
            d = self.target[l] - self.cur[l]
            self.cur[l] += max(-self.rate, min(self.rate, d))
        return CycleCommand(legs={l: LegStep(self.cur[l], 0.0, 0.10) for l in LEG_NAMES})
```

10.4 Verifying a command headlessly (detect IK clamping and multi-leg swing)

No PyBullet needed. Compare the commanded foot position to what the IK can actually reach by round-tripping through FK:

```
from Inverse_Kinematics.IK_A1 import (CrawlGait, LEG_NAMES, SIDE, L1, IK_X_SIGN,
                                      foot_to_joints, leg_fk)
def audit(cmd, seconds=None, dt=1/240, tol=1e-4):
    gait = CrawlGait(cmd, blend_time=0)                 # raw trajectory
    n = int((seconds or 2 * cmd.cycle_time) / dt)
    clamped = {l: 0.0 for l in LEG_NAMES}
    max_swing = 0
    for _ in range(n):
        gait.update(dt)
        max_swing = max(max_swing, len(gait.swinging))
        for l in LEG_NAMES:
            x, y, z = gait.foot_targets[l]
            side = SIDE[l]
            a, yy, zz = leg_fk(*foot_to_joints(l, x, y, z), side)
            err = max(abs(a - IK_X_SIGN * x), abs(yy - (side * L1 + y)), abs(zz - z))
            clamped[l] = max(clamped[l], err)
    for l, e in clamped.items():
        print(f"{l}: worst IK error {e*1000:6.2f} mm {'<-- UNREACHABLE' if e > tol else ''}")
    print("max simultaneous swing legs:", max_swing)
audit(CycleCommand.uniform(0.08, 0.0, 0.10))      # clean
audit(CycleCommand.uniform(0.30, 0.0, 0.10))      # shows clamping
```

Run `audit` over each cycle your provider will produce before putting it on hardware.

10.5 Live steering (closed-loop or keyboard)

A provider can be a callable object whose fields you modify while the sim runs; the values are sampled at each boundary:

```
class Steering:
    def __init__(self): self.speed, self.turn = 0.08, 0.0
    def __call__(self, n):
        return CycleCommand.differential(length_x=self.speed, turn=self.turn, height=0.10)
steer = Steering()
gait  = CrawlGait(steer(0), cycle_provider=steer)
# ... in the loop, from a key handler or controller:
steer.turn = 0.4        # takes effect at the next cycle boundary
```

For an immediate one-shot request (e.g. "stop after this cycle"):

```
gait.set_next_cycle(CycleCommand.uniform(length_x=0.0, height=0.10))
```

Note this still steps in place (feet lift by `height`). Use `height=0.0` and zero strides if you want the feet to stop lifting — though a crawl at `length=0` is effectively standing.

10.6 Circles and arbitrary planar paths

Reuse the kinematic idea from the circle script for any body twist `(vx, vy, ω)`: for each leg with neutral position `(fx, fy)` from the body centre,

```
def twist_command(vx, vy, omega, cycle_time=1.0, swing_frac=0.15, height=0.10,
                  hips=None, cg=-0.085):
    from Inverse_Kinematics.IK_A1 import SIDE, L1
    from circle_wlk_test import HIP_POS, CENTER
    stance_t = (1 - swing_frac) * cycle_time
    legs = {}
    for l in LEG_NAMES:
        hx, hy = HIP_POS[l]
        fx = hx + cg - CENTER[0]
        fy = hy + SIDE[l] * L1 - CENTER[1]
        legs[l] = LegStep((vx - omega * fy) * stance_t,
                          (vy + omega * fx) * stance_t, height)
    return CycleCommand(legs=legs, cycle_time=cycle_time,
                        swing_frac=swing_frac, cg_offset_x=cg)
```

Circle = `vx=v, vy=0, omega=v/R` (this reproduces `circle_command` when the strides are within limits). Remember to scale **all** legs together if any stride is too long.

10.7 Time-varying per-leg profiles

Since the provider receives the cycle index, per-leg values can follow any function:

```
def wobble(n):
    legs = {}
    for i, l in enumerate(LEG_NAMES):
        legs[l] = LegStep(length_x=0.07,
                          length_y=0.02 * math.sin(0.5 * n + i),
                          height=0.09 + 0.02 * (n % 2))
    return CycleCommand(legs=legs)
```

---

11. Intricacies and gotchas (checklist)

1. **Only** `length_x`, `length_y`, `height`, `phase_offset` **are per-leg.** `stance_z`, `cg_offset_x`, `swing_frac`, `cycle_time` are global. Adding per-leg stance depth requires storing `z`/`cg` per leg in `LegStep` and reading them in `_latch`.
2. **A new command needs a full cycle to take effect** on all legs (per-leg latching at own swing start). RL responds first.
3. `swing_frac`, `cycle_time`, `phase_offset` **are not latched** — changing them mid-run causes transients.
4. `swing_frac ≤ 0.25` for the one-leg-at-a-time crawl; it's a convention, not enforced.
5. **IK clamps silently** — validate with the `audit` helper. Symptoms: shortened strides, leg "sticking" in swing.
6. `IK_X_SIGN` **calibration**: test forward and backward once per URDF.
7. `Z_STAND`/`CG_X` **in the harness must equal** `stance_z`/`cg_offset_x` in your commands.
8. `update()` **returns a reused dict** — copy before storing.
9. `with_leg` **shares** `phase_offset` with the source command.
10. `neutral` **for the start blend is fixed at construction.**
11. **Uniform scaling for stride limits** (never clip a single leg) or the radius/path changes.
12. **Stale names** in docstrings/messages (`a1_ik.py`, `walk_test.py`, `walk_circle.py`).
13. **Negative** `height` drives the foot into the ground; use `0` for a sliding step.
14. **The gait is open-loop/kinematic:** there is no dynamic weight shift, terrain feedback or state estimation. `cg_offset_x` is a *static* shift; heavier payloads or slopes need retuning.
15. **Timestep:** the harness runs at `DT = 1/240 s` and the gait uses the `dt` you pass. Use a consistent `dt`; very large values skip through swing windows coarsely.
16. **Fall detection** in the demos is base height only (`z < 0.12`).

---

12. Tuning and troubleshooting

Symptom	Likely cause	Fix
Walks backward for positive `length_x`	`IK_X_SIGN` wrong	Flip to `+1`
Sidesteps the wrong way	Sign of `length_y` reversed in your logic	`+y` = left; check your convention
Strides shorter than commanded	IK clamping	Run `audit`; reduce stride or change `cg_offset_x`
Robot tips forward/back at lift-off	`cg_offset_x` too small/large	Tune around −0.06…−0.10 (more negative shifts weight forward)
Two legs in air, robot collapses	`swing_frac > 0.25` or bad offsets	`swing_frac ≤ 0.25`, keep default offsets

Symptom	Likely cause	Fix
Jerk when parameters change	Changed `swing_frac`/`phase_offset` mid-run	Change only at rest, or leave constant
Persistent yaw drift on "straight" walk	Asymmetric leg friction/strides	Trim `length_x` per side (10.2)
Feet stub on ground	`height` too low for terrain/speed	Raise `height` (check reach)
Motor saturation / jitter	Swing too fast (`cycle_time` small, stride large)	Raise `cycle_time`, lower strides, check `MAX_FORCE`
Circle fit radius wrong	Strides scaled or sign issue	Read the printed effective speed; check `IK_X_SIGN` and `length_y` sign

---

13. API reference

Constants: `L1, L2, L3, MAX_REACH, LEG_NAMES=("FR","FL","RR","RL"), SIDE, IK_X_SIGN, DEFAULT_PHASE_OFFSET`

Functions

- `leg_ik(x, y, z, side_sign, l1, l2, l3) -> (q1, q2, q3)` — IK-frame foot position relative to the hip; clamps to workspace.
- `leg_fk(q1, q2, q3, side_sign, ...) -> (x, y, z)` — inverse of `leg_ik` (IK frame).
- `foot_to_joints(leg, dx, dy, z) -> (q1, q2, q3)` — physical frame wrapper (applies `IK_X_SIGN` and `SIDE·L1`).

Classes

- `LegStep(length_x, length_y, height)` — frozen dataclass.
- `CycleCommand(legs, cycle_time, stance_z, swing_frac, cg_offset_x, phase_offset)` with `.uniform(...)`, `.differential(...)`, `.with_leg(leg, **kw)`.
- `CrawlGait(command, cycle_provider=None, blend_time=0.5)` with `.update(dt)`, `.set_next_cycle(cmd)`, `.swinging`, `.t`, `.cycle_index`, `.cycle_phase`, `.cmd`, `.foot_targets`, `.joint_targets`.

Self-test: `python IK_A1.py` runs a 2000-sample IK→FK round-trip check and a 6-second gait run that asserts (a) no per-tick foot jump beyond a few mm and (b) at most one swinging leg.

Harness helpers (`IK_module_test.py`): `DT, MAX_FORCE, Z_DOWN, Z_STAND, CG_X, DEMOS, leg_joint_map, command_legs, crouch_targets, step, stand_up, yaw_of`.
Circle helpers (`circle_wlk_test.py`): `HIP_POS, CENTER, circle_command, fit_circle`.